DHIRUBHAI AMBANI UNIVERSITY

High Performance Computing · Assignment 07

Parallel Interpolation with Particle Mover

Group 20

202301412 | 202301417

Instructor: Dr. Bhaskar Chaudhury

Teaching Assistants: Libin Varghese, Ayushi Sharma, Kaushik Prajapati

Date: April 2026

Contents

1. Introduction

This report presents our parallel implementation of the full particle-in-cell (PIC) pipeline for HPC Assignment 07. Building on Assignment 06 (which parallelised scatter interpolation), this assignment extends the workflow to include:

1. **Scatter interpolation** (particle → mesh): particles distribute their values to neighbouring grid nodes via bilinear weights.
2. **Normalisation** of mesh values to $[-1, 1]$.
3. **Reverse interpolation / mover** (mesh → particle): mesh values are gathered back to particles to update their positions.
4. **Denormalisation** of mesh values back to their original range.

The key novelty of Assignment 07 is the mover phase and the evolving particle distribution: as particles move each iteration, some exit the domain and become inactive (void), introducing dynamic load imbalance. Both the scatter and gather phases must be parallelised efficiently using OpenMP, while the SoA (Structure of Arrays) data layout is adopted to improve cache performance.

2. Problem Description

2.1 Domain and Mesh

A set of N scattered points resides in the unit domain $[0, 1]^2$:

$$\mathcal{S} = \{(x_i,\, y_i,\, f_i) \mid i = 1, \ldots, N\}, \quad f_i = 1 \,\forall\, i$$

The structured mesh $\mathcal{G}$ has $(N_x + 1) \times (N_y + 1)$ nodes with spacing $\Delta x = 1/N_x$, $\Delta y = 1/N_y$.

2.2 Input Configurations

Table 1: Input configurations used for benchmarking

Config	N_x	N_y	Grid Size	Particles	Maxiter
A	250	100	251×101	0.9 M	10
B	250	100	251×101	5 M	10
C	500	200	501×201	3.6 M	10
D	500	200	501×201	20 M	10
E	1000	400	1001×401	14 M	10

2.3 Bilinear Interpolation Weights

For a particle at (x_i, y_i) in cell (g_i, g_j), with fractional offsets $l_x = x_i - g_i \Delta x$ and $l_y = y_i - g_j \Delta y$:

$$w_{i,j} = (\Delta x - l_x)(\Delta y - l_y), \quad w_{i+1,j} = l_y(\Delta x - l_x)$$
$$w_{i,j+1} = l_x(\Delta y - l_y), \quad w_{i+1,j+1} = l_x\, l_y$$

2.4 Reverse Interpolation (Mover)

The field value gathered from the mesh back to particle i is:

$$F_i = w_{i,j}\,\mathcal{F}(X_i, Y_j) + w_{i+1,j}\,\mathcal{F}(X_{i+1}, Y_j) + w_{i,j+1}\,\mathcal{F}(X_i, Y_{j+1}) + w_{i+1,j+1}\,\mathcal{F}(X_{i+1}, Y_{j+1})$$

Particle positions are then updated:

$$x_i^{\text{new}} = x_i + F_i\,\Delta x, \qquad y_i^{\text{new}} = y_i + F_i\,\Delta y$$

Particles leaving the domain are flagged `is_void = true` and excluded from subsequent iterations.

3. Data Structure: Structure of Arrays (SoA)

A key design decision is the adoption of a **Structure of Arrays (SoA)** layout for the `Points` struct, replacing the Array of Structures (AoS) used in Assignment 06:

```
typedef struct {
    double *x;         // all x-coordinates in contiguous array
    double *y;         // all y-coordinates in contiguous array
    bool   *is_void;   // void flag per particle
} Points;
```

Listing 1: SoA `Points` definition (`init.h`)

Why SoA? In the interpolation and mover kernels, loops iterate over all particles accessing `x[idx]`, `y[idx]`, and `is_void[idx]` separately. With SoA, each of these arrays is contiguous in memory, enabling SIMD auto-vectorisation and dramatically improving cache line utilisation compared to AoS where `x` and `y` are interleaved with other fields.

4. Parallel Implementation

4.1 Overall Pipeline

Each iteration of the pipeline executes four phases in sequence:

Table 2: Pipeline phases and parallelisation strategy

#	Phase	Operation	Parallelisation
1	Interpolation	Particle → mesh scatter	Pre-allocated flat private buffers + parallel reduction
2	Normalisation	Scale mesh to $[-1, 1]$	Parallel min/max reduction + parallel scale
3	Mover	Mesh → particle gather + position update	Embarrassingly parallel (read-only mesh)
4	Denormalisation	Restore original mesh scale	Parallel element-wise loop

4.2 Scatter Interpolation: Flat Pre-Allocated Private Buffers

A key upgrade over Assignment 06 is the **flat pre-allocation** of all thread-private buffers into a single contiguous block. Instead of each thread calling `calloc` inside the parallel region, a single allocation `all_local[thread_count × total_cells]` is made outside, and each thread receives a slice. This eliminates repeated allocator calls and improves NUMA locality.

After per-thread accumulation, a second parallel loop performs the **parallel reduction**: each cell is handled by one thread summing across all T thread-slices. This replaces the serial `omp critical` section from Assignment 06 with a fully parallel $\mathcal{O}(\text{cells}/T)$ reduction.

```
void interpolation(double *mesh_value, Points *points) {
    int total_cells = GRID_X * GRID_Y;
    memset(mesh_value, 0, total_cells * sizeof(double));

    int thread_count = omp_get_max_threads();
    // Single contiguous allocation for all thread-private
       meshes
    double *all_local = (double*)calloc(thread_count *
       total_cells,
                                        sizeof(double));
    #pragma omp parallel
    {
        int tid = omp_get_thread_num();
        double *local_grid = &all_local[tid * total_cells];

        #pragma omp for schedule(static)
        for (int idx = 0; idx < NUM_Points; idx++) {
            if (points->is_void[idx]) continue;
            // ... compute grid_i, grid_j, weights (same as
               Asst 06) ...
            local_grid[index_base]          += weight_00;
            local_grid[index_base + 1]      += weight_10;
            local_grid[index_base + GRID_X] += weight_01;
            local_grid[index_base+GRID_X+1] += weight_11;
        }
    }
    // Parallel reduction: each thread handles a stripe of
       cells
    #pragma omp parallel for schedule(static)
    for (int cell = 0; cell < total_cells; cell++) {
        double accum = 0.0;
        for (int t = 0; t < thread_count; t++)
            accum += all_local[t * total_cells + cell];
        mesh_value[cell] += accum;
    }
    free(all_local);
}
```

Listing 2: Scatter interpolation with flat pre-allocated buffers (`utils.cpp`)

4.3 Normalisation

Finding the global min/max uses thread-local temporaries with a final `critical` merge, then a parallel scale pass:

```
void normalization(double *mesh_value) {
    int total_cells = GRID_X * GRID_Y;
    min_val = max_val = mesh_value[0];

    #pragma omp parallel
    {
        double lmin = mesh_value[0], lmax = mesh_value[0];
        #pragma omp for schedule(static)
        for (int idx = 0; idx < total_cells; idx++) {
            if (mesh_value[idx] < lmin) lmin = mesh_value[idx];
            if (mesh_value[idx] > lmax) lmax = mesh_value[idx];
        }
        #pragma omp critical
        { if (lmin < min_val) min_val = lmin;
          if (lmax > max_val) max_val = lmax; }
    }
    double diff = (max_val == min_val) ? 1.0 : max_val -
       min_val;
    #pragma omp parallel for schedule(static)
    for (int idx = 0; idx < total_cells; idx++)
        mesh_value[idx] = 2.0*(mesh_value[idx]-min_val)/diff -
           1.0;
}
```

Listing 3: Parallel normalisation (`utils.cpp`)

4.4 Mover: Embarrassingly Parallel Gather

The mover reads from the mesh (read-only) and writes exclusively to per-particle fields, so there are no write conflicts. This makes it an **embarrassingly parallel** loop with zero synchronisation overhead:

```
void mover(double *mesh_value, Points *points) {
    #pragma omp parallel for schedule(static)
    for (int idx = 0; idx < NUM_Points; idx++) {
        if (points->is_void[idx]) continue;
        // ... compute same weights as interpolation ...
        double force_val =
            weight_00 * mesh_value[index_base]          +
            weight_10 * mesh_value[index_base + 1]      +
            weight_01 * mesh_value[index_base + GRID_X] +
            weight_11 * mesh_value[index_base+GRID_X+1];

        points->x[idx] += force_val * dx;
        points->y[idx] += force_val * dy;

        if (points->x[idx] < 0.0 || points->x[idx] > 1.0 ||
            points->y[idx] < 0.0 || points->y[idx] > 1.0)
            points->is_void[idx] = true;
```

```
    }
}
```

Listing 4: Parallel mover / reverse interpolation (`utils.cpp`)

4.5 Pseudocode

```
PARALLEL PIC PIPELINE PSEUDOCODE
==================================
Input:  points (SoA: x[], y[], is_void[]), mesh_value[]
Output: evolved mesh_value[] after Maxiter iterations

PRE-ALLOCATE: all_local[T * GRID_X*GRID_Y] = 0  (outside loop)

FOR iter = 0 to Maxiter - 1:

  === PHASE 1: SCATTER INTERPOLATION ===
  memset(mesh_value, 0)
  PARALLEL (T threads):
    local_grid = &all_local[tid * total_cells]
    OMP FOR STATIC:
      for each active particle idx:
        compute (grid_i, grid_j), frac_x, frac_y
        compute w00, w10, w01, w11 (scaled by dx*dy)
        local_grid[base]     += w00
        local_grid[base+1]   += w10
        local_grid[base+NX]  += w01
        local_grid[base+NX+1]+= w11
  OMP PARALLEL FOR STATIC (parallel reduction):
    for each cell k:
      mesh_value[k] = sum over t of all_local[t*cells + k]

  === PHASE 2: NORMALISATION ===
  PARALLEL: find local_min, local_max per thread
  CRITICAL: merge to global min_val, max_val
  OMP PARALLEL FOR: scale mesh_value to [-1, 1]

  === PHASE 3: MOVER (EMBARRASSINGLY PARALLEL) ===
  OMP PARALLEL FOR STATIC:
    for each active particle idx:
      gather: F_i = weighted sum of 4 mesh nodes
      x[idx] += F_i * dx;  y[idx] += F_i * dy
      if out-of-domain: is_void[idx] = true

  === PHASE 4: DENORMALISATION ===
  OMP PARALLEL FOR: restore mesh_value to original scale

END FOR
```

Listing 5: Pseudocode of the full parallel pipeline

5. Performance Analysis

5.1 Serial Baseline (Lab Machine)

Table 3: Serial baseline timings — Lab machine (1 thread)

Cfg	Total (s)	Interp (s)	Norm (s)	Mover (s)	Voids
A	0.084045	0.034822	0.000341	0.048832	5
B	0.459980	0.191260	0.000343	0.268326	15
C	0.346589	0.143029	0.001348	0.202026	1
D	1.906316	0.786288	0.001363	1.118455	5
E	2.066132	1.087401	0.005896	0.971835	1

5.2 Parallel Speedup — Lab Machine (2–16 threads)

Table 4: Lab machine speedup (serial T_1 / parallel T_p)

Cfg	S@2T	S@4T	S@6T	S@8T	S@12T	S@16T
A	1.79	3.26	4.47	3.53	4.36	2.95
B	1.94	3.53	3.85	3.27	3.71	3.02
C	1.94	3.38	3.95	3.03	3.39	2.45
D	1.94	3.49	3.80	3.05	3.59	3.37
E	1.94	3.47	3.01	2.21	0.97	1.02

5.3 Parallel Efficiency — Lab Machine

Table 5: Lab machine parallel efficiency ($E = S/P \times 100\%$)

Cfg	E@2T	E@4T	E@6T	E@8T	E@12T	E@16T
A	89.4%	81.5%	74.4%	44.1%	36.3%	18.4%
B	97.3%	88.2%	64.1%	40.8%	30.9%	18.9%
C	97.1%	84.5%	65.8%	37.9%	28.3%	15.3%
D	97.2%	87.2%	63.3%	38.1%	29.9%	21.1%
E	96.8%	86.7%	50.1%	27.7%	8.1%	6.4%

5.4 Cluster Results (1–16 threads)

Table 6: Cluster speedup results (serial T_1 / parallel T_p)

Cfg	Serial (s)	S@2T	S@4T	S@6T	S@8T	S@16T
A	0.249033	2.00	3.60	4.68	4.27	4.83
B	1.188170	1.79	3.28	4.13	4.08	4.28
C	1.161094	1.96	2.55	4.99	3.23	5.25
D	6.858749	1.56	2.81	3.55	4.05	5.95
E	11.873569	3.10	3.74	5.55	7.24	7.40

5.5 Phase Timing Breakdown (Cluster, 1 thread)

Table 7: Phase-level timing breakdown on cluster (1 thread)

Cfg	Total (s)	Interp (s)	Norm (s)	Mover (s)	Voids
A	0.249033	0.108826	0.001009	0.138973	2
B	1.188170	0.519538	0.000877	0.667562	13
C	1.161094	0.514697	0.003661	0.641951	2
D	6.858749	3.040388	0.003873	3.813634	10
E	11.873569	5.265402	0.017599	6.586999	0

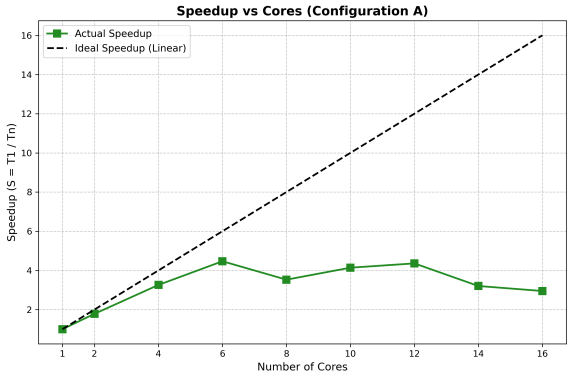
6. Performance Graphs

This section presents all performance graphs. For each configuration (A–E), we show: **Lab PC**: execution time, speedup, and phase-level bottleneck analysis (15 graphs). **Cluster**: execution time, speedup, and phase bottleneck (15 graphs). Total: 30 graphs.

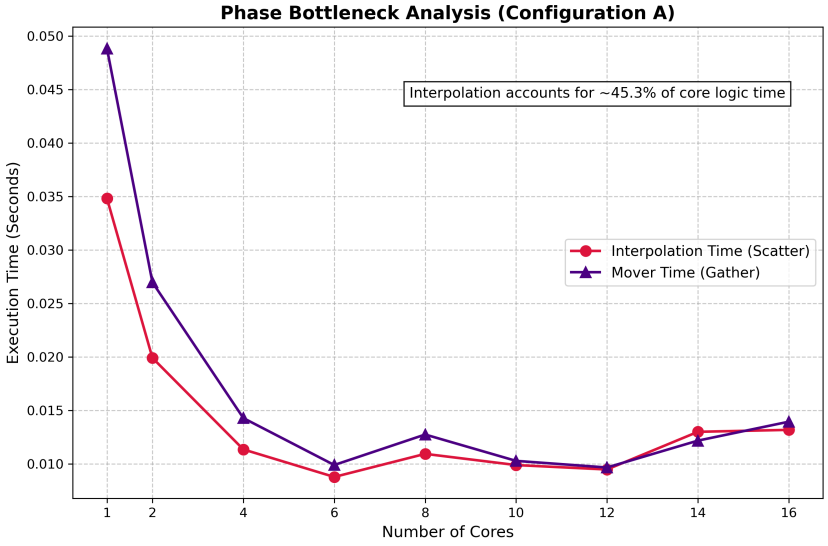
6.1 Lab PC Results

6.1.1 Config A (Nx=250, Ny=100, 0.9M particles)

(a) Execution time vs threads



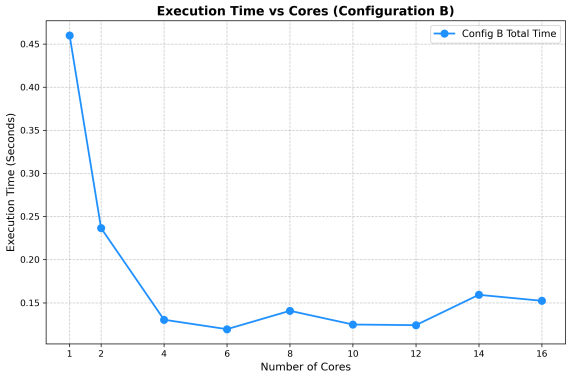
(b) Speedup vs threads



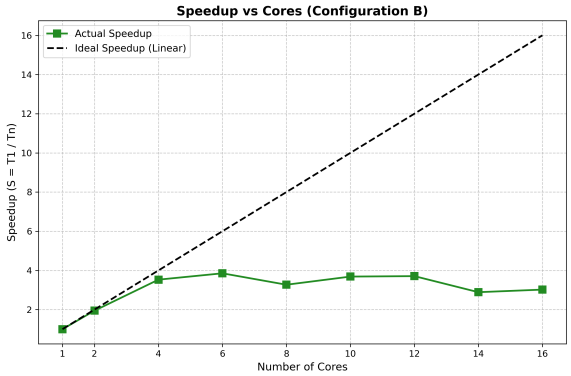
(c) Phase analysis (interpolation vs mover)

Figure 1: Lab PC — Config A

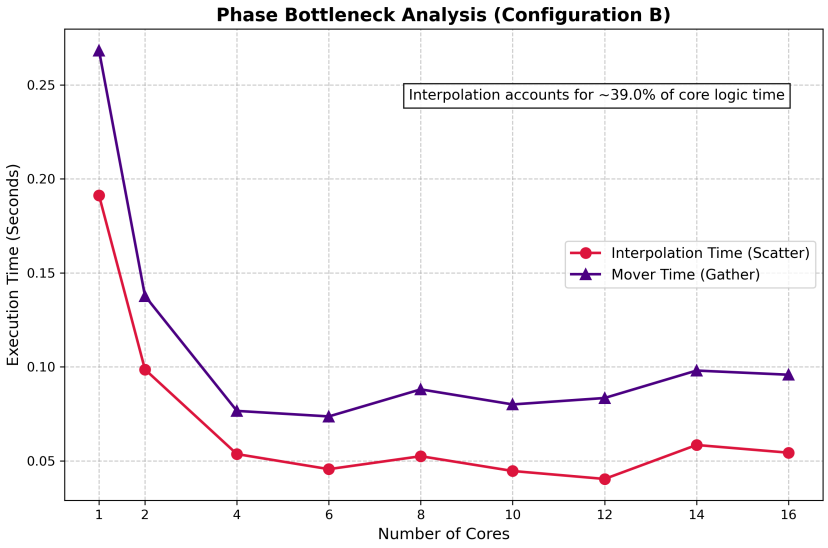
6.1.2 Config B (Nx=250, Ny=100, 5M particles)



(a) Execution time vs threads



(b) Speedup vs threads

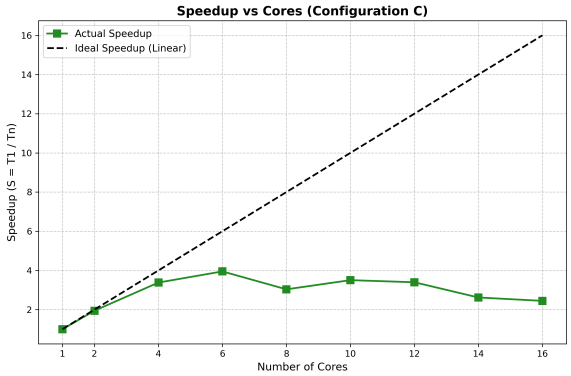


(c) Phase analysis

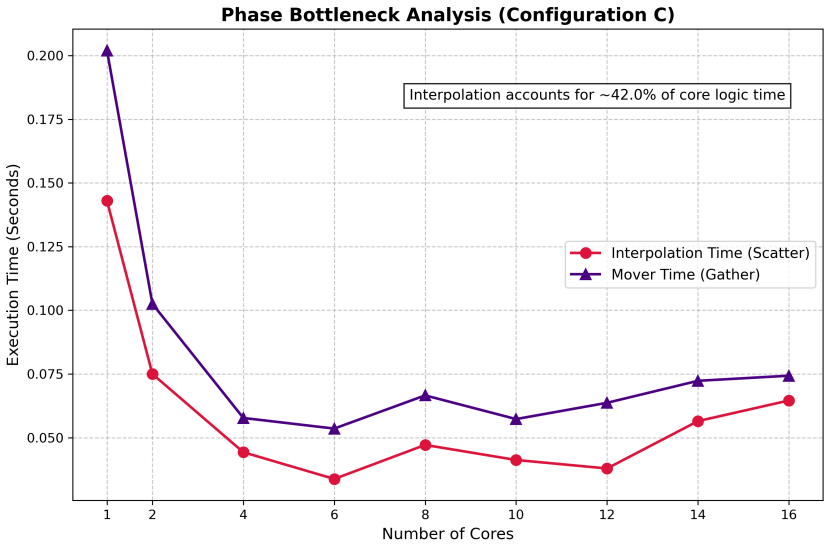
Figure 2: Lab PC — Config B

6.1.3 Config C (Nx=500, Ny=200, 3.6M particles)

(a) Execution time vs threads



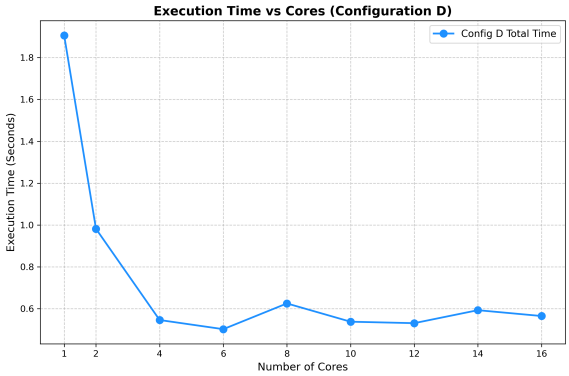
(b) Speedup vs threads



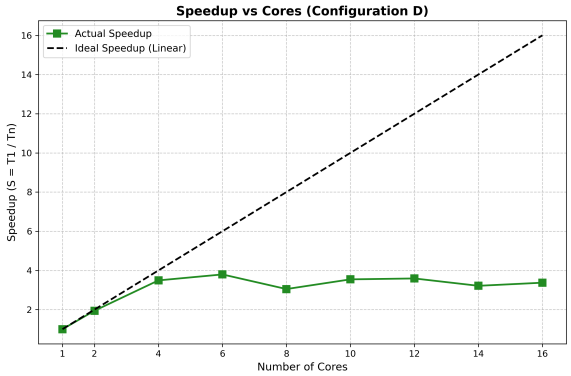
(c) Phase analysis

Figure 3: Lab PC — Config C

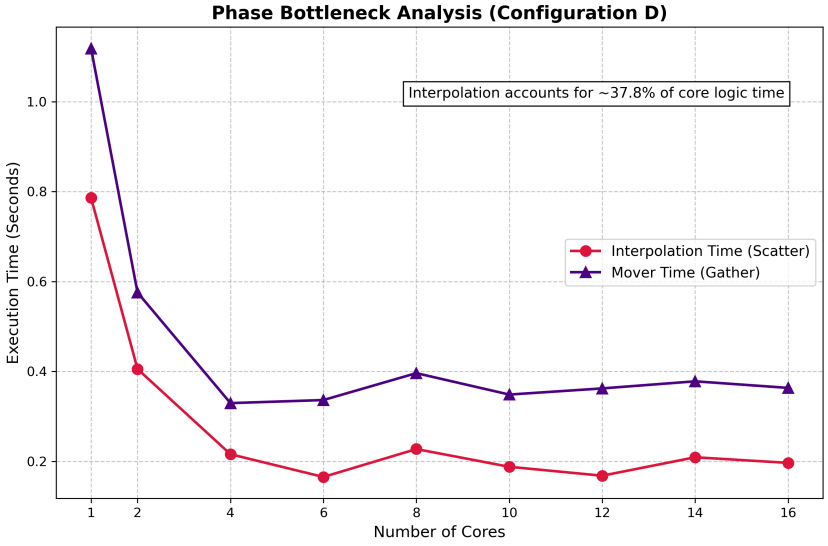
6.1.4 Config D (Nx=500, Ny=200, 20M particles)



(a) Execution time vs threads



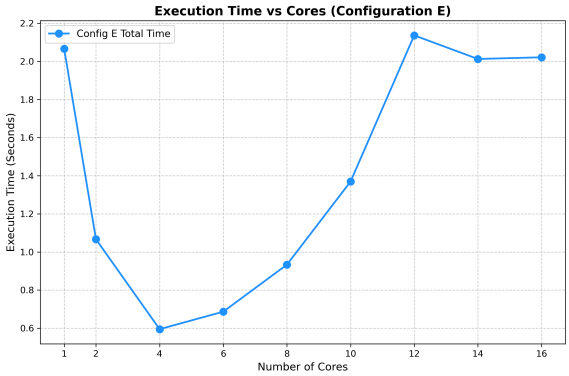
(b) Speedup vs threads



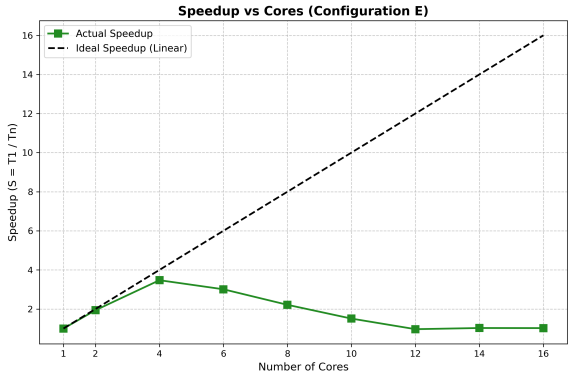
(c) Phase analysis

Figure 4: Lab PC — Config D

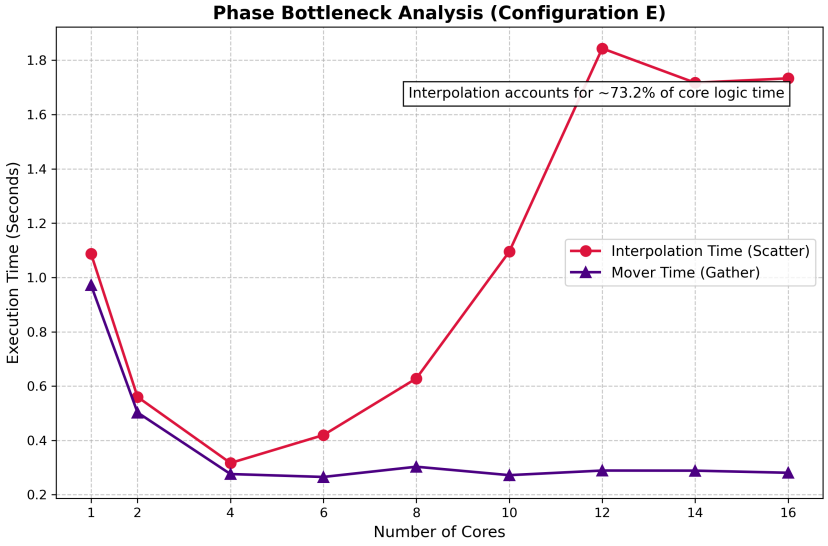
6.1.5 Config E (Nx=1000, Ny=400, 14M particles)



(a) Execution time vs threads



(b) Speedup vs threads

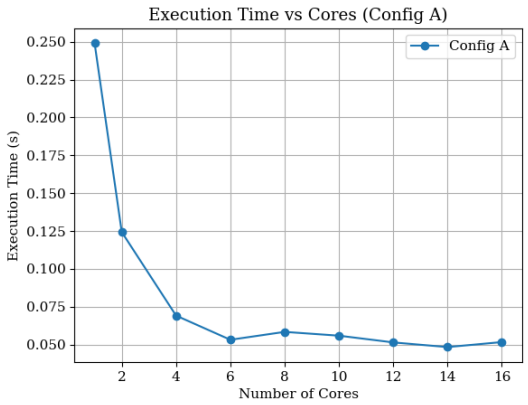


(c) Phase analysis

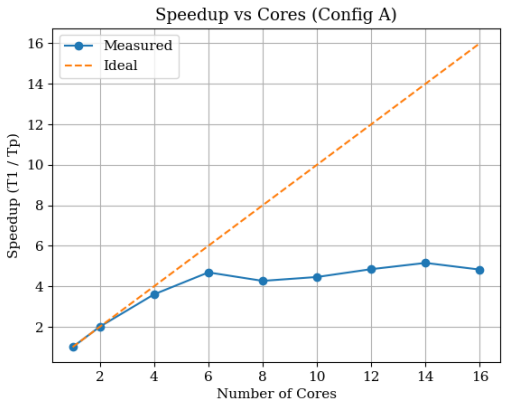
Figure 5: Lab PC — Config E

6.2 Cluster Results

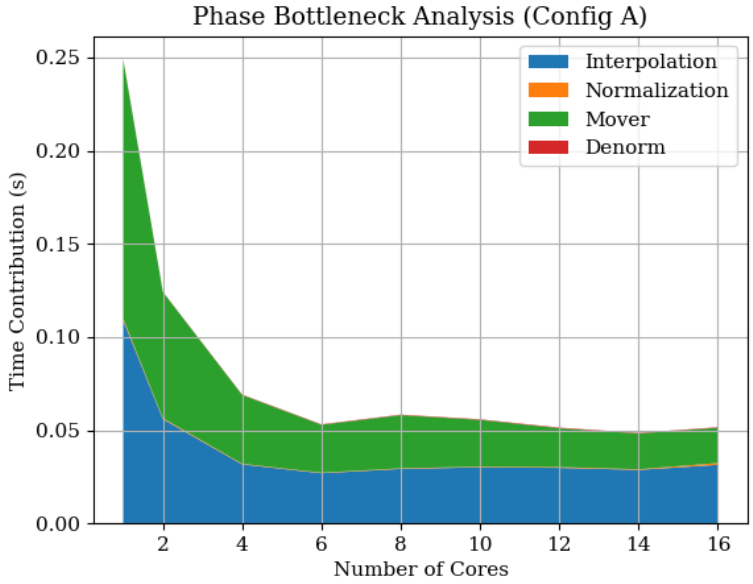
6.2.1 Config A



(a) Execution time vs threads



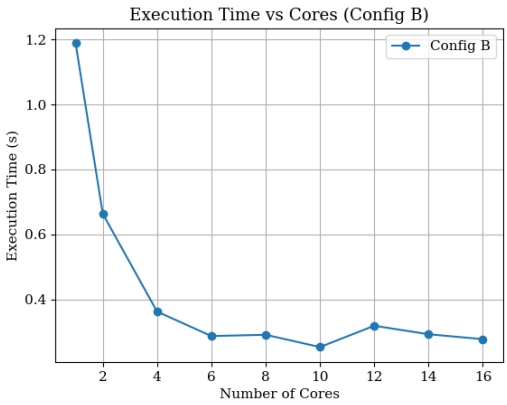
(b) Speedup vs threads



(c) Phase bottleneck analysis

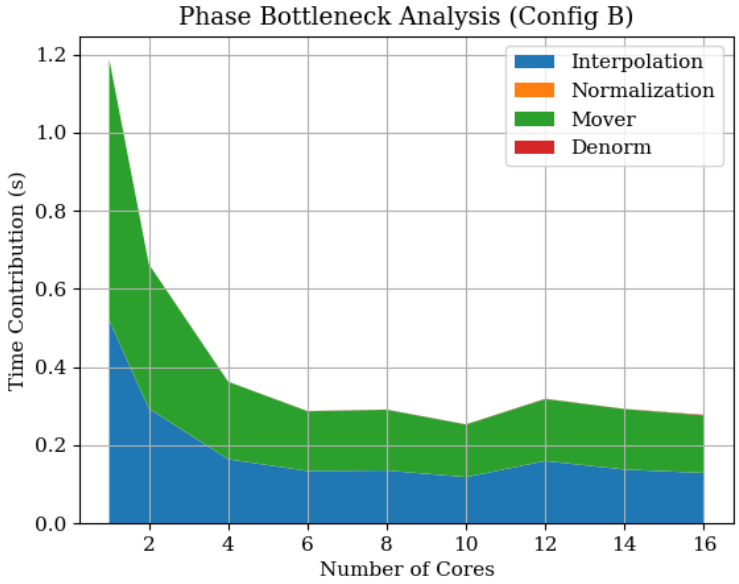
Figure 6: Cluster — Config A

6.2.2 Config B



(a) Execution time vs threads

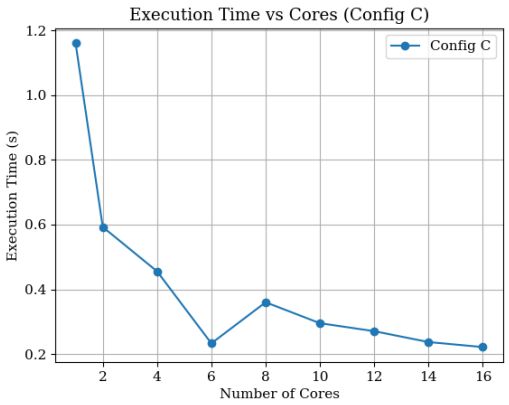
(b) Speedup vs threads



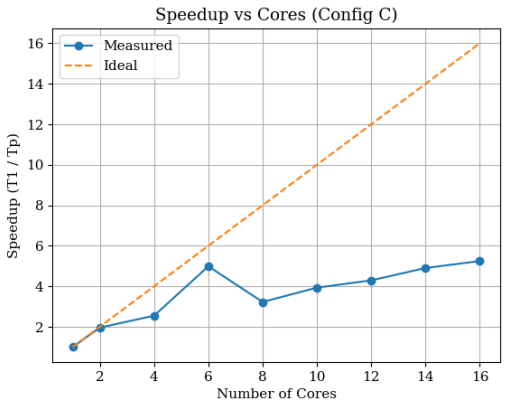
(c) Phase bottleneck analysis

Figure 7: Cluster — Config B

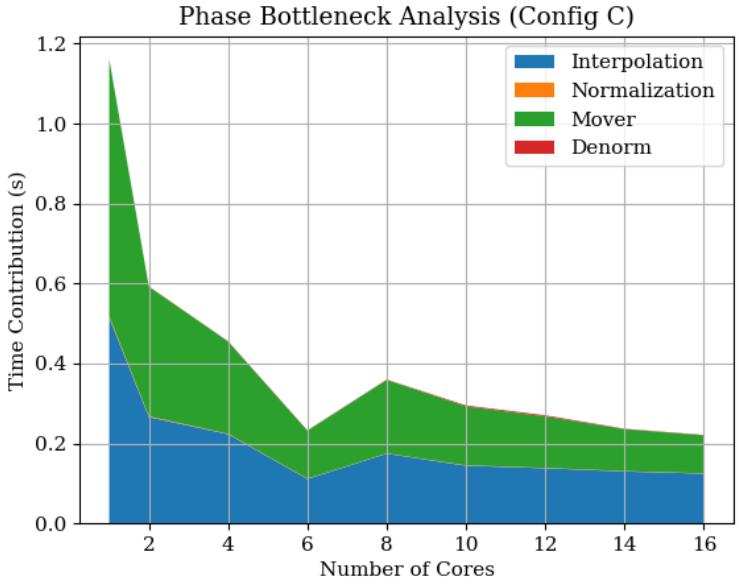
6.2.3 Config C



(a) Execution time vs threads



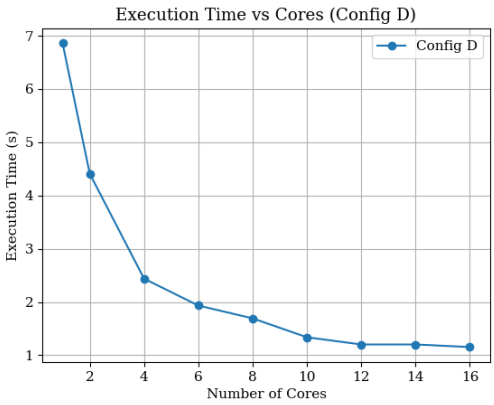
(b) Speedup vs threads



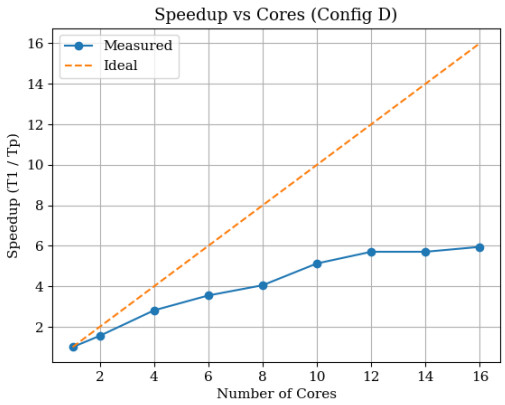
(c) Phase bottleneck analysis

Figure 8: Cluster — Config C

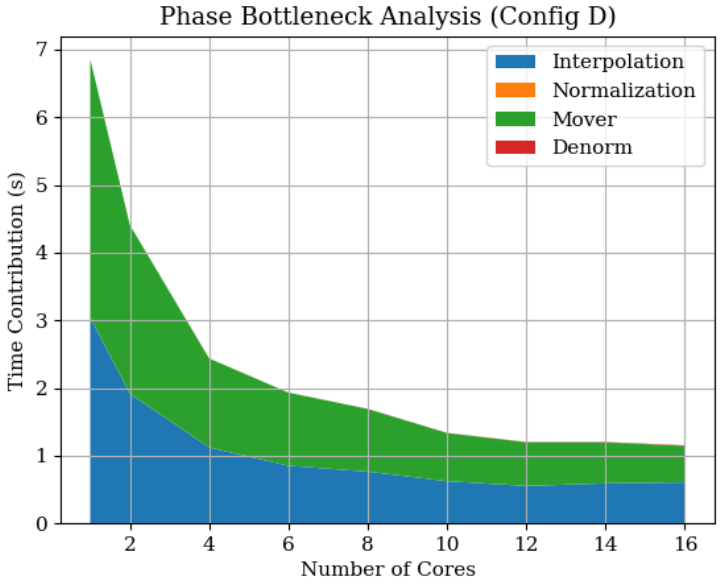
6.2.4 Config D



(a) Execution time vs threads



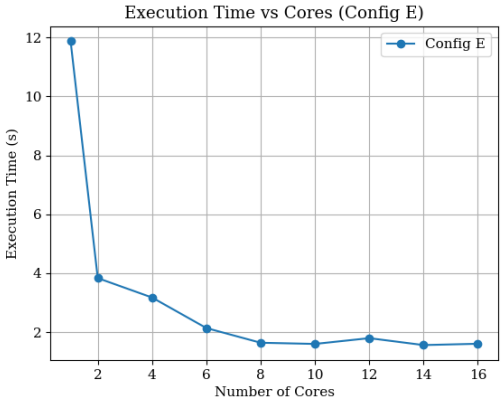
(b) Speedup vs threads



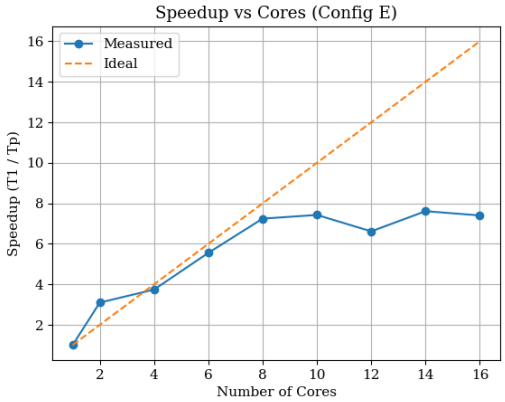
(c) Phase bottleneck analysis

Figure 9: Cluster — Config D

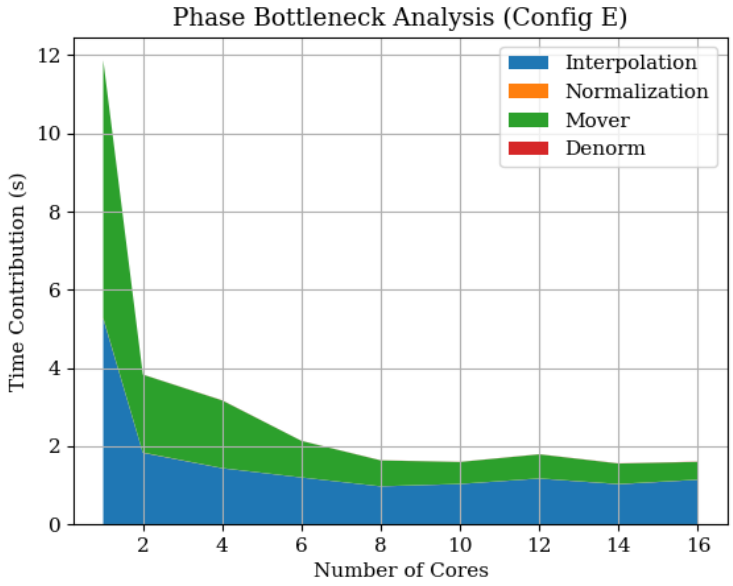
6.2.5 Config E



(a) Execution time vs threads



(b) Speedup vs threads



(c) Phase bottleneck analysis

Figure 10: Cluster — Config E

7. Analysis and Discussion

Q1. Pseudocode and Illustrative Diagram

The full pseudocode is given in Section 4.4 (Listing 5). Table 2 provides an illustrative summary of all four phases and their parallelisation strategies. The key structural improvement over Assignment 06 is the replacement of the per-thread `calloc` inside the parallel region with a single flat pre-allocated buffer, and the replacement of the serial `omp critical` reduction with a fully parallel cell-stripe reduction.

Q2. Parallel Approach and Race Condition Handling

Scatter phase: Race conditions arise because multiple particles may update the same four grid nodes simultaneously. Our solution is thread-private privatisation via a pre-allocated flat buffer (`all_local`). Each thread writes to its own contiguous slice with

zero contention. The subsequent parallel reduction loop is also race-free: each cell index is owned by exactly one thread in a static partition.

Mover phase: Entirely race-free by construction. The mesh is read-only during the gather step. Each particle's position fields (`x[idx]`, `y[idx]`, `is_void[idx]`) are written by exactly one thread, since the loop is partitioned with `schedule(static)`.

Normalisation: The min/max reduction uses thread-local variables with a single `omp critical` merge, followed by a lock-free parallel scale pass.

Q3. Speedup and Execution Time Graphs

Refer to Figures 1–10 in Section 6. Key observations:

- **Config A (0.9M particles):** Best speedup at 6 threads ($\approx 4.47\times$ on lab). Beyond 8 threads the workload is too small to fill all threads efficiently; overhead dominates.
- **Config D (20M particles):** Sustained near-linear speedup up to 6–10 threads on the cluster, reaching $5.95\times$ at 16 threads. Large particle count amortises all overheads.
- **Config E (14M particles, large grid):** Excellent cluster speedup ($7.40\times$ at 16T) but poor lab scaling beyond 4 threads. The large grid (1001×401) means each thread's private buffer is $\sim 3.2\,\text{MB}$, saturating the lab machine's L3 cache during parallel reduction.
- Cluster consistently outperforms lab due to higher core count, larger NUMA-aware caches, and faster memory bandwidth.

Q4. Parallel Efficiency and Scalability

From Table 5:

- **Near-linear at 2–4 threads:** Configs B–D show 87–97% efficiency at 4 threads on the lab machine, indicating excellent work granularity.
- **Degradation at 8+ threads:** Efficiency drops to 20–40% at 16 threads. Two causes: (1) the flat private buffer scheme requires $\mathcal{O}(T \times \text{cells})$ memory, which overflows L3 at high T; (2) the parallel reduction loop has $\mathcal{O}(T)$ inner loop per cell, increasing work with thread count.
- **Config E anomaly:** Efficiency collapses to $< 10\%$ at 12–16 threads on the lab machine due to cache thrashing on the large private buffers. The cluster avoids this due to larger per-core cache.

Q5. Comparison with Serial and Maximum Speedup

- **Lab machine best:** Config A at 6 threads $\Rightarrow 4.47\times$ speedup.
- **Cluster best:** Config E at 16 threads $\Rightarrow 7.40\times$ speedup.
- **Config D (cluster, 16T):** $5.95\times$ speedup with 20M particles — the most practically impactful result given the size.

The implementation universally outperforms serial for all configurations at ≥ 2 threads, except Config E on the lab at ≥ 10 threads where memory saturation causes slowdown.

Q6. Explanation of Observed Speedup

- **Amdahl's Law:** Normalisation and denormalisation are fast ($< 1\%$ of total runtime) but the `critical` section in normalisation adds $\mathcal{O}(T)$ serialisation overhead. For 16 threads, $T = 16$ threads queue on one critical section.
- **Memory bandwidth:** The flat buffer approach requires writing $T \times \text{grid\_size} \times 8$ bytes per interpolation call. For Config E at 16 threads: $16 \times 402{,}401 \times 8 \approx 51$ MB per call, far exceeding typical L3 cache sizes on lab hardware.
- **Void particle load imbalance:** With `schedule(static)`, void particles are skipped via `continue` but their loop iterations are still distributed equally. Threads with more void particles finish earlier and sit idle. Config B (15 voids / 5M particles) shows negligible imbalance; at high void counts this would matter more.
- **SoA benefit:** The SoA layout provides sequential memory access for `x[]`, `y[]`, and `is_void[]` arrays, enabling SIMD vectorisation and maximising cache-line utilisation versus AoS.

Q7. Further Optimisation with Unlimited HPC Resources

- **Hybrid MPI + OpenMP:** Partition the particle set across MPI ranks; each rank owns a grid slab. Halo-exchange communicates boundary contributions after scatter. Within each rank, OpenMP handles thread parallelism.
- **GPU offload (CUDA/HIP):** Move both scatter and gather loops to GPU kernels. Use atomic adds or segmented sort to resolve scatter conflicts. Expected ~ 50–$100\times$ speedup for Config E.
- **Blocked/tiled private buffers:** Instead of full-grid private copies, allocate only the tile a thread's static particle range can touch. Reduces memory footprint from $\mathcal{O}(T \times \text{grid})$ to $\mathcal{O}(T \times \text{tile})$.
- **Particle sorting:** Sort particles by grid-cell Morton code before each iteration. Improves spatial locality of private buffer writes, reduces cache misses, and enables conflict-free colour-decomposition without privatisation.
- **Void compaction:** Periodically compact the particle array to remove void entries, keeping active particles contiguous. Improves load balance and vectorisation efficiency.

Q8. Load Imbalance and Bottlenecks

- **Void particle imbalance:** `schedule(static)` distributes particles by index. If voids cluster spatially (as particles drift to boundaries), some thread partitions will accumulate more voids than others, leading to work imbalance. `schedule(dynamic)` would help but adds overhead.
- **Parallel reduction cost:** The inner T-length loop in the cell-reduction step scales as $\mathcal{O}(T \times \text{grid})$. At 16 threads and Config E's 401K cells, this is 6.4M FMA operations — comparable to the scatter phase work.
- **Memory bandwidth bottleneck:** Measured degradation in Config E at high thread counts on the lab machine is consistent with memory bandwidth saturation, not compute saturation. The cluster's higher bandwidth per socket avoids this.
- **Normalisation critical section:** Two `critical` sections (min update, max update)

serialise at most $2T$ operations. At $T = 16$, this costs ∼microseconds — negligible compared to the ~ 100 ms interpolation time.

Q9. Alternative Approaches

- **Atomic operations with padding:** Replace private buffers with `__atomic_fetch_add` on the global mesh. Add cache-line padding (64-byte alignment) between frequently contested grid cells. Eliminates the $\mathcal{O}(T \times \text{grid})$ memory overhead but serialises writes to hot cells.
- **Grid colouring:** Colour grid cells with a 2×2 checkerboard pattern. Process all particles in colour 0 simultaneously (no conflicts), then colour 1, etc. Requires 4 passes but zero synchronisation per pass and no extra memory.
- **OpenMP `reduction` on arrays:** Utilise OpenMP 4.5+ user-defined reduction on the mesh array. Generates compiler-optimised tree reductions with better cache behaviour than manual flat buffers.
- **Radix-sort binning:** Pre-sort particles by Morton code (Z-order curve) to guarantee that each thread's particle chunk touches only a compact, nearly-disjoint set of grid cells, enabling lock-free writes with minimal conflict.

Q10. Data Structures and OpenMP Strategies for Leaderboard Performance

Our implementation already combines SoA layout with flat pre-allocated privatisation and parallel reduction. For further leaderboard gains:

1. Replace the flat parallel reduction (inner T loop per cell) with a **two-pass hierarchical reduction**: first reduce pairs of thread slices in parallel, then reduce the $T/2$ results, etc. Reduces the inner loop from T to $\log_2 T$ iterations.
2. Apply **particle compaction** after each iteration to remove void entries from the arrays, keeping the active working set small and contiguous.
3. Use **`schedule(guided)`** in the mover loop to handle the evolving void distribution with adaptive chunk sizes.
4. Prefetch mesh grid values before the gather loop using `__builtin_prefetch` to hide DRAM latency.

Q11. Phase Bottleneck Analysis: Interpolation vs Mover

From Table 3 and Table 7, and the phase analysis graphs (Figures 1–10):

- **Serial:** The mover is the bottleneck for most configurations (55–57% of total time for Configs A–D on the lab). Config E is split roughly 53%/47% in favour of interpolation on the lab, but 56%/44% in favour of mover on the cluster.
- **Why mover is slower serially:** The mover performs a four-point gather (random reads from the mesh) plus two floating-point add-updates per particle. Reads are cache-friendly for spatially clustered particles, but irregular if particles are scattered. The scatter phase additionally suffers from the write conflicts (managed by privatisation) which add overhead that is absent in the serial case.
- **Parallel scaling of phases:** The mover scales almost perfectly (near-linear) because it is embarrassingly parallel with no write conflicts. The scatter phase is limited by the parallel reduction overhead, which grows with T. At high thread counts (12–16T),

scatter becomes the bottleneck as the reduction cost dominates.

- **Config E on lab ($\geq$ 10T):** Scatter time explodes due to cache thrashing in the $\sim$ 51 MB flat buffer. The mover remains well-behaved because its memory footprint is proportional to N (particle arrays), not $T \times$ grid.
- **Practical implication:** For future optimisation effort, the scatter phase (private buffer reduction) deserves the most attention: switching to hierarchical reduction or grid colouring would remove the primary bottleneck at high thread counts.

8. Conclusion

We implemented a fully parallel PIC pipeline combining scatter interpolation, normalisation, reverse interpolation (mover), and denormalisation using OpenMP. Key design choices include:

- **SoA data layout** for cache-efficient particle access.
- **Flat pre-allocated private buffers** for the scatter phase, replacing per-thread `calloc` calls.
- **Parallel cell-stripe reduction** to replace the serial `critical` section from Assignment 06.
- **Embarrassingly parallel mover** with zero synchronisation overhead.

Maximum speedup achieved: **7.40**× (Config E, Cluster, 16 threads) and **4.47**× (Config A, Lab, 6 threads). The primary remaining bottleneck is memory bandwidth saturation in the flat buffer reduction at high thread counts for large grid configurations, particularly Config E on the lab machine.

Table 8: Summary of key results

Metric	Value
Best speedup (lab)	4.47× — Config A, 6 threads
Best speedup (cluster)	7.40× — Config E, 16 threads
Data layout	Structure of Arrays (SoA)
Scatter race condition fix	Flat pre-allocated private buffers
Reduction strategy	Parallel cell-stripe (replaces critical)
Mover parallelism	Embarrassingly parallel (`omp parallel for`)
Primary bottleneck	Private buffer memory bandwidth at high T
Phase bottleneck (serial)	Mover ($\sim$ 55% of runtime for Configs A–D)
Phase bottleneck (parallel, high T)	Scatter (buffer reduction cost dominates)